

42

Complete Setup Guide — v5.2

42 School · System Administration Project

Debian

Rocky

VirtualBox

VM Setup · LVM · SSH · UFW · sudo · Password Policy · monitoring.sh · WordPress

Table of Contents

1	Virtual Machine Setup	VM creation in VirtualBox / UTM
2	OS Installation — Debian	Minimal install, no GUI
3	LVM Encrypted Partitioning	At least 2 encrypted partitions
4	Network & SSH Configuration	Port 4242, no root login
5	UFW Firewall	Only port 4242 open
6	User & Group Management	user42, sudo groups
7	Password Policy	chage + PAM pwquality
8	sudo Configuration	Strict rules + logging
9	monitoring.sh Script	System info via wall every 10 min
10	Cron Scheduling	Run monitoring.sh automatically
11	Bonus — WordPress Stack	lighttpd + MariaDB + PHP
12	Submission & Signature	signature.txt + README.md
13	Defense Cheat Sheet	Key concepts & commands

1 Virtual Machine Setup

■ INFO

You must use VirtualBox (or UTM on Apple Silicon). No graphical interface is allowed — installing X.org or Wayland results in grade 0.

1.1 Creating the VM in VirtualBox

- Open VirtualBox → click New
- Name: your_login42 | Type: Linux | Version: Debian (64-bit)
- RAM: 1024 MB (minimum, 2048 recommended)
- Hard disk: Create a virtual hard disk now → VDI → Dynamically allocated → 12 GB (30 GB for bonus)
- Do NOT add a graphical controller — server only

Setting	Value
Name	yourlogin42 (e.g. wil42)
Type	Linux
Version	Debian GNU/Linux (64-bit)
RAM	1024 MB – 2048 MB
Storage	12 GB VDI dynamically allocated
Network	NAT (default) → can change to Bridged for SSH testing
Display	No 3D, no additional displays

1.2 Port Forwarding (for SSH from host)

Go to VM Settings → Network → Advanced → Port Forwarding:

Name	Protocol	Host Port	Guest Port
SSH	TCP	4242	4242

2 OS Installation — Debian

■ INFO

Download the latest stable Debian ISO from [debian.org](https://www.debian.org). Mount it as a virtual CD in VirtualBox before starting the VM.

2.1 Installer Steps

- Boot from ISO → select Install (NOT Graphical Install)
- Language: English | Country: your country | Keymap: your layout
- Hostname: yourlogin42 (e.g. wil42)
- Domain name: leave blank
- Root password: set a strong password (policy applies later)
- Create a non-root user with your login name
- Partitioning: Manual (see Section 3)
- Software selection: UNCHECK everything including 'Debian desktop environment'
- → Only keep 'SSH server' and 'standard system utilities'
- Install GRUB to the primary drive

■ WARNING

Do NOT select any desktop environment. No Gnome, KDE, XFCE. The machine must be text-only.

```
Debian Installer

$ # Boot screen – choose:
$ # Install (NOT Graphical Install)

$ # Software selection screen – UNCHECK:
$ # [ ] Debian desktop environment
$ # [ ] GNOME
$ # [*] SSH server          <-- keep this
$ # [*] standard system utilities <-- keep this
```

3 LVM Encrypted Partitioning

■ INFO

You must create at least 2 encrypted partitions using LVM. The mandatory layout uses /boot (unencrypted) + an LVM group inside an encrypted partition.

3.1 Mandatory Partition Layout

```
root@wil42:~#
```

```
$ lsblk
```

NAME	MAJ:MIN	SIZE	TYPE	MOUNTPOINT
sda	8:0	12G	disk	
■sda1	8:1	487M	part	/boot
■sda2	8:2	1K	part	
■sda5	8:5	7.5G	part	
■sda5_crypt	254:0	7.5G	crypt	
■wil--vg-root	254:1	2.8G	lvm	/
■wil--vg-swap	254:2	976M	lvm	[SWAP]
■wil--vg-home	254:3	3.8G	lvm	/home

3.2 Bonus Partition Layout

```
root@wil42:~# (Bonus)
```

```
$ lsblk
```

NAME	SIZE	TYPE	MOUNTPOINT
sda	30.8G	disk	
■sda1	500M	part	/boot
■sda2	1K	part	
■sda5	30.3G	part	
■sda5_crypt	30.3G	crypt	
■LVMGroup-root	10G	lvm	/
■LVMGroup-swap	2.3G	lvm	[SWAP]
■LVMGroup-home	5G	lvm	/home
■LVMGroup-var	3G	lvm	/var
■LVMGroup-srv	3G	lvm	/srv
■LVMGroup-tmp	3G	lvm	/tmp
■LVMGroup-var-log	4G	lvm	/var/log

3.3 Step-by-Step Partitioning (Debian Installer)

1. In the installer choose: Partitioning method → Manual
2. Select the disk (sda) → Create new empty partition table
3. Create /boot: 500 MB, primary, ext4, mount /boot
4. Create remaining space as logical, leave as physical volume for encryption

5. Configure encrypted volumes → create encrypted volume on the partition
6. Set passphrase (remember this!)
7. Configure the LVM: create volume group 'LVMGroup' on encrypted device
8. Create logical volumes: root (10G), swap (2.3G), home (5G), etc.
9. Set filesystems: ext4 for most, swap for swap partition
10. Finish partitioning and write changes to disk

✓ **TIP**

During defense, you can verify with: `lsblk` and `cat /etc/fstab`

4

Network & SSH Configuration

■ INFO

SSH must run on port 4242 only. Root login via SSH must be disabled.

4.1 Install & Configure OpenSSH

```
root@wil42:~#  
  
$ sudo apt update && sudo apt install openssh-server -y  
$ sudo vim /etc/ssh/sshd_config  
  
# --- Edit these lines ---  
Port 4242  
PermitRootLogin no  
PasswordAuthentication yes  
  
$ sudo systemctl restart ssh  
$ sudo systemctl enable ssh  
$ sudo systemctl status ssh
```

4.2 Verify SSH is Listening

```
root@wil42:~#  
  
$ ss -tunlp | grep 4242  
tcp LISTEN 0 128 0.0.0.0:4242 0.0.0.0:* users:(("sshd",pid=523))  
tcp LISTEN 0 128 [::]:4242 [::]:* users:(("sshd",pid=523))
```

4.3 Connect from Host Machine

```
host@machine:~$  
  
$ ssh yourlogin@127.0.0.1 -p 4242  
# Or if using bridged networking:  
$ ssh yourlogin@192.168.x.x -p 4242
```

■ WARNING

Root SSH login must be DISABLED. Test this: `ssh root@127.0.0.1 -p 4242` should be rejected.

5 UFW Firewall Configuration

■ INFO

Only port 4242 must be open. UFW must be active at VM startup. Rocky uses firewalld instead.

5.1 Install & Configure UFW (Debian)

```
root@wil42:~#  
  
$ sudo apt install ufw -y  
$ sudo ufw default deny incoming  
$ sudo ufw default allow outgoing  
$ sudo ufw allow 4242  
$ sudo ufw enable  
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y  
Firewall is active and enabled on system startup  
  
$ sudo ufw status verbose  
Status: active  
To Action From  
--  
4242 ALLOW IN Anywhere  
4242 (v6) ALLOW IN Anywhere (v6)
```

5.2 firewalld Configuration (Rocky)

```
root@rocky42:~#  
  
$ sudo firewall-cmd --permanent --add-port=4242/tcp  
success  
$ sudo firewall-cmd --reload  
success  
$ sudo firewall-cmd --list-ports  
4242/tcp  
$ sudo firewall-cmd --state  
running
```

✓ TIP

Make sure ufw is enabled BEFORE you disconnect SSH, or you might lock yourself out.
Keep your VirtualBox console open as backup.

6

User & Group Management

■ INFO

You need a non-root user with your login. This user must belong to both 'user42' and 'sudo' groups.

6.1 Create Groups and Assign User

```
root@wil42:~#  
  
# Create the user42 group  
$ sudo groupadd user42  
  
# Add your user to user42 and sudo groups  
$ sudo usermod -aG user42 yourlogin  
$ sudo usermod -aG sudo yourlogin  
  
# Verify group membership  
$ groups yourlogin  
yourlogin : yourlogin sudo user42  
  
# Or check with id command  
$ id yourlogin  
uid=1000(yourlogin) gid=1000(yourlogin) groups=1000(yourlogin),27(sudo),1001(user42)
```

6.2 During Peer Review — Create a New User

```
root@wil42:~#  
  
# Create new user (peer review task)  
$ sudo adduser newuser  
Adding user 'newuser' ...  
Enter new UNIX password:  
  
# Add to a group  
$ sudo adduser newuser user42  
  
# Verify  
$ groups newuser  
newuser : newuser user42
```

7

Password Policy

Rule	Value	Config File
Expiry	Every 30 days	/etc/login.defs
Min days between changes	2 days	/etc/login.defs
Warning before expiry	7 days	/etc/login.defs
Min length	10 characters	PAM pwquality
Uppercase required	Yes	PAM pwquality
Lowercase required	Yes	PAM pwquality
Number required	Yes	PAM pwquality
Max consecutive identical	3	PAM pwquality
Must not contain username	Yes	PAM pwquality
New chars vs old password	7 minimum (non-root)	PAM pwquality

7.1 Edit /etc/login.defs

```
root@wil42:~#  
  
$ sudo vim /etc/login.defs  
  
# Find and change these lines:  
PASS_MAX_DAYS    30  
PASS_MIN_DAYS    2  
PASS_WARN_AGE    7
```

7.2 Install & Configure PAM pwquality

```
root@wil42:~#  
  
$ sudo apt install libpam-pwquality -y  
  
$ sudo vim /etc/pam.d/common-password  
  
# Find the line with pam_pwquality.so and edit it to:  
password requisite pam_pwquality.so retry=3 \  
    minlen=10 ucredit=-1 lcredit=-1 dcredit=-1 \  
    maxrepeat=3 reject_username difok=7 enforce_for_root
```

7.3 Apply Policy to Existing Accounts

```
root@wil42:~#  
  
# Apply expiry settings to existing users  
$ sudo chage --maxdays 30 --mindays 2 --warndays 7 yourlogin  
$ sudo chage --maxdays 30 --mindays 2 --warndays 7 root  
  
# Verify settings  
$ sudo chage -l yourlogin  
Last password change          : Jan 01, 2025  
Password expires              : Jan 31, 2025  
Password inactive             : never  
Minimum number of days       : 2  
Maximum number of days       : 30  
Number of days of warning    : 7  
  
# Change all passwords after config  
$ sudo passwd yourlogin  
$ sudo passwd root
```

8

sudo Configuration

■ INFO

sudo must be configured with strict rules: 3-attempt limit, custom error message, full logging, TTY mode, and restricted paths.

8.1 Install sudo

```
root@wil42:~#  
  
$ su -  
Password:  
root@wil42:~# apt install sudo -y  
root@wil42:~# usermod -aG sudo yourlogin  
root@wil42:~# exit
```

8.2 Configure sudoers

```
root@wil42:~#  
  
# ALWAYS use visudo to edit sudoers (syntax check):  
$ sudo visudo  
  
# Add these lines at the bottom:  
Defaults    passwd_tries=3  
Defaults    badpass_message="Wrong password! 3 tries max."  
Defaults    logfile="/var/log/sudo/sudo.log"  
Defaults    log_input  
Defaults    log_output  
Defaults    iolog_dir="/var/log/sudo/"  
Defaults    requiretty  
Defaults    secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"
```

8.3 Create sudo log directory

```
root@wil42:~#  
  
$ sudo mkdir -p /var/log/sudo  
$ sudo touch /var/log/sudo/sudo.log  
$ sudo chmod 700 /var/log/sudo  
  
# Test logging  
$ sudo ls /root  
$ sudo cat /var/log/sudo/sudo.log
```

■ WARNING

Never edit `/etc/sudoers` directly — always use `'sudo visudo'`. A syntax error can lock you out of sudo completely.

9 monitoring.sh Script

■ INFO

Create a bash script that broadcasts system info to all terminals every 10 minutes using 'wall'. No errors should be displayed.

9.1 Script Location & Creation

```
root@wil42:~#
$ sudo vim /usr/local/bin/monitoring.sh
$ sudo chmod +x /usr/local/bin/monitoring.sh
```

9.2 Full monitoring.sh Script (Part 1 — Variables)

```
monitoring.sh – Part 1

#!/bin/bash
# Architecture & kernel
ARCH=$(uname -a)
# Physical CPUs
PCPU=$(grep -c 'physical id' /proc/cpuinfo 2>/dev/null)
[ -z "$PCPU" ] && PCPU=1
# Virtual CPUs
VCPU=$(nproc)
# RAM usage
RAM_TOTAL=$(free -m | awk '/^Mem:/{print $2}')
RAM_USED=$(free -m | awk '/^Mem:/{print $3}')
RAM_PCT=$(free | awk '/^Mem:/{printf("%.2f"), $3/$2*100}')
# Disk usage
DISK_TOTAL=$(df -Bg | grep '^/dev/' | grep -v '/boot' | awk '{ft+=$2} END{print ft}')
DISK_USED=$(df -Bm | grep '^/dev/' | grep -v '/boot' | awk '{fu+=$3} END{print fu}')
DISK_PCT=$(df -Bm | grep '^/dev/' | grep -v '/boot' | awk '{fu+=$3;ft+=$4} END{printf("%d"), fu/(fu+ft)}')
# CPU load
CPU_LOAD=$(top -bn1 | grep '^%Cpu' | awk '{printf "%.1f", $2+$4}')
# Last boot
LAST_BOOT=$(who -b | awk '{print $3" "$4}')
# LVM active?
LVM=$(lsblk | grep -q 'lvm' && echo yes || echo no)
# TCP established connections
TCP=$(ss -ta | grep ESTAB | wc -l)
# Users logged in
USERS=$(users | wc -w)
# Network info
IP=$(hostname -I | awk '{print $1}')
MAC=$(ip link show | grep 'ether' | awk '{print $2}')
# Sudo command count
SUDO=$(journalctl _COMM=sudo | grep COMMAND | wc -l)
```

Part 2 — wall broadcast

```
monitoring.sh – Part 2

wall "
#Architecture:    $ARCH
#Physical CPU:    $PCPU
#vCPU:            $VCPU
#Memory Usage:    $RAM_USED/${RAM_TOTAL}MB ($RAM_PCT%)
#Disk Usage:      $DISK_USED/${DISK_TOTAL}Gb ($DISK_PCT%)
#CPU load:        $CPU_LOAD%
#Last boot:       $LAST_BOOT
#LVM use:         $LVM
#TCP Connections: $TCP ESTABLISHED
#User log:        $USERS
#Network:         IP $IP ($MAC)
#Sudo:            $SUDO cmd"
```

9.3 Expected Output

```
wall broadcast

Broadcast message from root@wil42 (tty1) (Sun Apr 25 15:45:00 2025):

#Architecture: Linux wil42 6.1.0-18-amd64 #1 SMP Debian x86_64 GNU/Linux
#Physical CPU: 1
#vCPU: 1
#Memory Usage: 74/987MB (7.50%)
#Disk Usage: 1009/2Gb (49%)
#CPU load: 6.7%
#Last boot: 2025-04-25 14:45
#LVM use: yes
#TCP Connections: 1 ESTABLISHED
#User log: 1
#Network: IP 10.0.2.15 (08:00:27:51:9b:a5)
#Sudo: 42 cmd
```

10 Cron Scheduling

■ INFO

Use cron to run monitoring.sh every 10 minutes at startup. During peer review, you must be able to stop it WITHOUT modifying the script.

10.1 Add Cron Job

```
root@wil42:~#  
  
# Edit root's crontab  
$ sudo crontab -u root -e  
  
# Add this line (runs every 10 minutes):  
@reboot sleep 10 && bash /usr/local/bin/monitoring.sh  
*/10 * * * * bash /usr/local/bin/monitoring.sh  
  
# Verify  
$ sudo crontab -u root -l  
*/10 * * * * bash /usr/local/bin/monitoring.sh
```

10.2 Stop the Script Without Modifying It

During the defense, the evaluator may ask you to stop the broadcast. Do this by commenting out the cron job:

```
root@wil42:~#  
  
# Open crontab  
$ sudo crontab -u root -e  
  
# Comment out the line with #:  
# */10 * * * * bash /usr/local/bin/monitoring.sh  
  
# Save and exit – no restart needed  
# The script is NOT modified, only the schedule is disabled
```

Cron Field Reference

Field	Values	Example
Minute	0 – 59	*/10 = every 10 min
Hour	0 – 23	*/1 = every hour
Day of month	1 – 31	* = every day
Month	1 – 12	* = every month
Day of week	0 – 7 (0=Sun)	* = every day

11 Bonus — WordPress Stack

■ INFO

Bonus: Set up WordPress with lighttpd + MariaDB + PHP. NGINX and Apache2 are EXCLUDED. Bonus is only evaluated if the mandatory part is PERFECT.

11.1 Install lighttpd

```
root@wil42:~#  
  
$ sudo apt install lighttpd -y  
$ sudo systemctl enable lighttpd  
$ sudo systemctl start lighttpd  
$ sudo ufw allow 80  
  
# Enable CGI and FastCGI modules  
$ sudo lighty-enable-mod fastcgi  
$ sudo lighty-enable-mod fastcgi-php  
$ sudo service lighttpd force-reload
```

11.2 Install MariaDB

```
root@wil42:~#  
  
$ sudo apt install mariadb-server -y  
$ sudo systemctl enable mariadb  
$ sudo mysql_secure_installation  
  
# Follow prompts:  
# Remove anonymous users? Y  
# Disallow root login remotely? Y  
# Remove test database? Y  
  
# Create WordPress database  
$ sudo mysql -u root -p  
MariaDB [(none)]> CREATE DATABASE wordpress;  
MariaDB [(none)]> CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'StrongP@ss1';  
MariaDB [(none)]> GRANT ALL ON wordpress.* TO 'wpuser'@'localhost';  
MariaDB [(none)]> FLUSH PRIVILEGES;  
MariaDB [(none)]> EXIT;
```

11.3 Install PHP & WordPress

```
root@wil42:~#  
  
$ sudo apt install php-cgi php-mysql php-curl php-gd php-mbstring -y  
  
# Download WordPress  
$ cd /var/www/html  
$ sudo wget https://wordpress.org/latest.tar.gz  
$ sudo tar -xvzf latest.tar.gz  
$ sudo mv wordpress/* .  
$ sudo rm -rf wordpress latest.tar.gz  
  
# Configure wp-config.php  
$ sudo cp wp-config-sample.php wp-config.php  
$ sudo vim wp-config.php  
# Set DB_NAME='wordpress', DB_USER='wpuser', DB_PASSWORD='StrongP@ss1'  
  
$ sudo chown -R www-data:www-data /var/www/html  
$ sudo chmod -R 755 /var/www/html
```

12 Submission & Signature

■ INFO

Submit only: signature.txt and README.md at the root of your Git repository. NEVER commit the VM file itself.

12.1 Generate the Signature

```
host@machine:~$  
  
# On Linux host:  
$ sha1sum ~/VirtualBox\ VMs/wil42/wil42.vdi  
6e657c4619944be17df3c31faa030c25e43e40af  wil42.vdi  
  
# On MacOS:  
$ shasum ~/VirtualBox\ VMs/wil42/wil42.vdi  
  
# On Mac M1 (UTM):  
$ shasum ~/Library/Containers/com.utmapp.UTM/Data/Documents/wil42.utm/Images/disk-0.qcow2  
  
# On Windows:  
$ certUtil -hashfile wil42.vdi sha1
```

12.2 Save the Signature

```
host@machine:~$  
  
# Copy the hash into signature.txt  
$ echo '6e657c4619944be17df3c31faa030c25e43e40af' > signature.txt  
  
# Commit to your 42 Git repo  
$ git add signature.txt README.md  
$ git commit -m 'Born2beRoot submission'  
$ git push
```

✗ DANGER

Your VM signature changes every time you start it. Generate the signature from a POWERED-OFF snapshot/clone. Each evaluation must start with NO snapshots.

12.3 README.md Required Sections

- First line (italicized): This project has been created as part of the 42 curriculum by
- Description: project goal, overview
- Instructions: how to run / install
- Resources: references + AI usage description
- Project description: OS choice (Debian vs Rocky), design choices, comparisons:

- → Debian vs Rocky Linux
- → AppArmor vs SELinux
- → UFW vs firewalld
- → VirtualBox vs UTM

13 Defense Cheat Sheet

■ INFO

Key concepts you must be able to explain. Understand what you use — not just copy commands.

13.1 Key Commands for Verification

root@wil42:~#

```
# Check OS & kernel
$ uname -a
$ head -n2 /etc/os-release

# Check AppArmor (Debian)
$ sudo aa-status
$ sudo systemctl status apparmor

# Check SELinux (Rocky)
$ sestatus

# Check partitions & LVM
$ lsblk
$ sudo lvdisplay

# Check UFW
$ sudo ufw status verbose

# Check SSH
$ sudo systemctl status ssh
$ ss -tunlp | grep 4242

# Check sudo config
$ sudo visudo
$ sudo cat /var/log/sudo/sudo.log

# Check password policy
$ sudo chage -l yourlogin
$ grep PASS /etc/login.defs

# Check groups
$ groups yourlogin
$ getent group sudo
$ getent group user42

# Check cron
$ sudo crontab -u root -l
```

13.2 Concept Comparison Table

Concept	Option A	Option B	Key Difference
OS	Debian	Rocky	Debian: apt, AppArmor, beginner-friendly. Rocky: dnf, SELinux, RHEL-based
Security	AppArmor	SELinux	AppArmor: path-based profiles, simpler. SELinux: label-based, stricter
Firewall	UFW	firewalld	UFW: simple rules. firewalld: zones, dynamic, Rich Rules
Hypervisor	VirtualBox	UTM	VirtualBox: x86 hosts. UTM: Apple Silicon (ARM), uses QEMU
Pkg mgr	apt	aptitude	apt: lower-level. aptitude: interactive UI, smarter dependency resolution

13.3 Quick Hostname Change (Peer Review)

```
root@wil42:~#  
  
# Change hostname during peer review  
$ sudo hostnamectl set-hostname newhostname42  
$ sudo vim /etc/hosts  
# Change: 127.0.1.1 oldname42 → 127.0.1.1 newhostname42  
  
# Verify (needs logout/login to refresh prompt)  
$ hostname  
newhostname42
```

✓ TIP

Before your defense: take a SHA1 of the powered-off VDI, save it to signature.txt, push to git, then ONLY work with snapshots or clones.